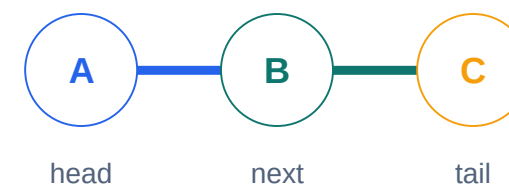


Связные списки. Стек, очередь и дек

Лекция 4 · 2 академических часа
03.03.01 Прикладные математика и физика

МОДЕЛЬ ТЕМЫ



LIFO ↔ FIFO

Почему известная позиция вставки спасает связный список, но не спасает массив?

Ответ требует связать представление узлов указателями, инвариант связности и стоимость операций у обоих концов последовательности.

После лекции вы сможете

ИНВАРИАНТ

Объяснять

Инварианты односвязного и двусвязного списка и контракты их операций.

Оценивать

Стоимость операций списка, стека, очереди и дека в худшем и амортизированном смысле.

Выбирать

Связный список, массив или кольцевой буфер для стека, очереди и дека под задачу.

Логика лекции: от указателя к инженерному выбору

МОДЕЛЬ

Как связаны узлы?

Что это стоит?

Что выбрать?



Указатель next (и prev)
вместо смежности ячеек.

$\Theta(1)$ рядом с известным
узлом, $\Theta(i)$ — при поиске по
индексу.

Связный список, кольцевой
буфер или готовый ADT-
контейнер.

Связный список и массив — два способа хранить порядок

Связный список

Порядок задают указатели, а не смежность.
Вставка по известному узлу — $\Theta(1)$.
Обход и доступ по индексу — $\Theta(n)$.

Массив (лекция 3)

Порядок задаёт смежность ячеек.
Доступ по индексу — $\Theta(1)$.
Вставка и удаление внутри — $\Theta(n)$.

Состояние узла и списка задают несколько полей

ИНВАРИАНТ

- 01 value — хранимое значение узла.
- 02 next — ссылка на следующий узел (и prev в двусвязном списке).
- 03 head — указатель на первый узел (и tail — на последний, если поддерживается).
- 04 Инвариант: next(последнего) = $\emptyset$; список конечен и не содержит циклов.

Контракт

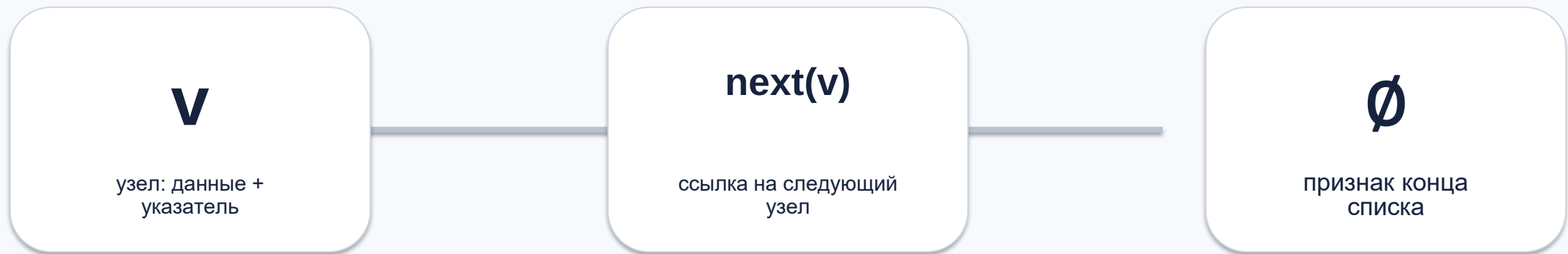
Пустой список — это не особый тип, а состояние $head = \emptyset$; операции обязаны оставаться корректными и для него.

Узел хранит данные и указатель — адрес вычислять не нужно

$\text{next}(v)$ — явная ссылка, а не результат арифметики над базовым адресом;
позиция в памяти не обязана быть смежной.

Каждый узел ссылается на следующий по указателю

МОДЕЛЬ



head $\rightarrow$ **v₁** $\rightarrow$ **v₂** $\rightarrow$... $\rightarrow$ **v_{k-1}** $\rightarrow$ $\emptyset$

$\Theta(1)$ вставки по узлу не отменяет предусловий

ИНВАРИАНТ

Предусловие

Ссылка v указывает на существующий узел списка L
Список L удовлетворяет инварианту
Значение x определено

Постусловие

Новый узел присоединён рядом с v корректным указателем
Остальные узлы и их порядок не изменились
Инвариант списка сохранён

Операции списка тоже имеют разную цену

МОДЕЛЬ

По известному узлу

INSERT_AFTER / DELETE_AFTER — $\Theta(1)$.
Меняются только соседние указатели.

По значению или индексу

Поиск и доступ — $\Theta(n)$.
Адресной формулы нет, нужен обход.

Связный список теряет то, что давал массив, — локальность

МОДЕЛЬ

Пространственная

Узлы связного списка обычно не соседствуют в памяти — `next` ведёт куда угодно.

Временная

Обход остаётся линейным $\Theta(n)$, но чаще промахивается мимо кэша, чем у массива.

Следствие

Одинаковая Θ -оценка не гарантирует одинаковое фактическое время — как и в лекции 3.

Известный узел не спасает от поиска по позиции

МОДЕЛЬ

Ссылка на узел есть

Вставка/удаление рядом — $\Theta(1)$.
Соседние указатели меняются напрямую.

Известен только номер

Нужен обход от head — $\Theta(i)$.
Сама вставка после этого — уже $\Theta(1)$.

Вставка после известного узла — четыре шага

МОДЕЛЬ

- 01 Убедиться, что v — существующий узел списка.
- 02 Создать новый узел u со значением x .
- 03 $u.next \leftarrow v.next$.
- 04 $v.next \leftarrow u$.

Важно

Порядок присваиваний обязателен: если сначала выполнить $v.next \leftarrow u$, ссылка на старый $v.next$ будет потеряна.

Стек и очередь — это контракт, а не структура

ИНВАРИАНТ

Стек (LIFO)

push/pop работают с одним концом.

Реализация —

INSERT_FRONT/DELETE_FRONT.

Очередь (FIFO)

enqueue добавляет в конец,

dequeue — из начала.

Без tail одна из операций теряет $\Theta(1)$.

Общее

Обе операции у выбранного конца — $\Theta(1)$ на связном списке.

Один интерфейс, два открытых конца — push/pop с обеих сторон

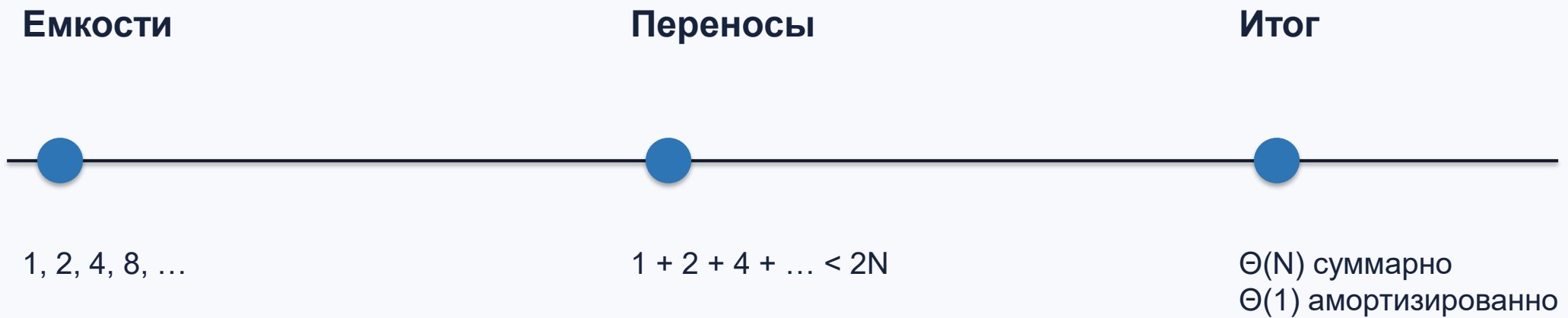
Дек обобщает стек и очередь: PUSH_FRONT, PUSH_BACK, POP_FRONT, POP_BACK обязаны быть $\Theta(1)$ в худшем случае.

Дек на двусвязном списке даёт $\Theta(1)$ на обоих концах без исключений

ВЫВОД



Дек на кольцевом буфере повторяет анализ динамического массива



Дек можно реализовать разными способами с одним контрактом

ИНВАРИАНТ

Двусвязный список
Кольцевой буфер
`collections.deque` / `std::deque`

Общая идея

Важен контракт $\Theta(1)$ на обоих концах, а не конкретное внутреннее устройство.

Локальность и память определяют, что предпочесть на практике

Выбор между связным списком и кольцевым буфером определяется профилем операций, а не только асимптотикой.

Компромисс присутствует и в реализации дека

МОДЕЛЬ

Двусвязный список

Нет амортизации, но выше память на элемент и хуже локальность.

Кольцевой буфер

Лучше локальность и память, но переполнение требует расширения.

Смешанный выбор

Готовые `collections.deque` / `std::deque` снимают этот выбор с прикладного кода.

Одна абстракция ADT — разные реализации на практике ПРОВЕРКА

Python collections.deque

Двусвязный список блоков фиксированного размера.
 $\Theta(1)$ амортизированно с обоих концов.

C++ std::deque

Карта фрагментов фиксированного размера.
Те же гарантии, другое внутреннее устройство.

Скользящий максимум на деке — $\Theta(N)$, а не $\Theta(N \cdot w)$

МОДЕЛЬ

```
SLIDING_MAX(x, N, w):  
    D ← empty_deque()  
    for i in 0..N-1:  
        pop_front, если front вне окна  
        pop_back, пока x[back] ≤ x[i]  
        push_back(D, i); взять x[front(D)]
```

Граница измерения

Каждый индекс входит в дек не более одного раза.

Суммарно $\Theta(N)$, а не $\Theta(N \cdot w)$.

Память дека растёт с w , а не с N .

Типичные ошибки со связными структурами

РИСК

- 01 Терять единственную ссылку на head при обходе.
- 02 Вставлять или удалять узел, не сохранив нужный next/prev заранее.
- 03 Забывать обновить tail при удалении последнего узла.
- 04 Путать порядок присваиваний указателей при развороте списка.
- 05 Использовать дек на кольцевом буфере без обработки переполнения.

Мини-задача: разворот короткого списка

ПРОВЕРКА

Дано

$L: 3 \rightarrow 7 \rightarrow 1 \rightarrow \emptyset$
Выполняется `REVERSE(L)`.

Обоснуйте

Состояние `prev` и `curr` после каждой итерации и итоговый `head` после завершения.

Проверьте себя перед следующей лекцией

ПРОВЕРКА

- 01 Почему вставка по известному узлу — $\Theta(1)$?
- 02 Зачем очереди нужен указатель tail?
- 03 Чем двусвязный список лучше односвязного для дека?
- 04 Почему дек на кольцевом буфере повторяет анализ лекции 3?
- 05 Меняется ли $\Theta(N)$ скользящего максимума при повторяющихся значениях x ?

Связная структура меняет прямую адресацию на указатели: дёшево у известного места, дорого — по значению или индексу

Следующая лекция: сравнительные алгоритмы сортировки и нижняя граница.